PLATAFORMA DE DATOS SEGURA Y AUTOMATIZADA PARA PYMES DE E-COMMERCE

CONVOCATORIA: 23 de febrero de 2026

ALUMNO/A: Alfonso Nieto García

TUTOR/A: Paola García Iturralde

CICLO: Administración de sistemas informáticos en red

INDICE

Contenido

1. ABSTRACT 3
2. INTRODUCCIÓN 3
   2.1 Justificación y contexto 3
   2.2 Planteamiento del problema 3
   2.3 Objetivos del trabajo 3
3. MOTIVACION 4
   3.1 Marco teórico del trabajo 4
   3.2 Otros desarrollos anteriores 5
   3.3 Análisis funcional del proyecto 5
      Contexto, alcance y actores 5
      Alcance del sistema 5
      Actores y roles 6
      Requisitos funcionales y no funcionales 6
      Funcionalidades detalladas y criterios de aceptación 8
      Interfaces del sistema y contrato de datos 8
      Interfaces internas 9
      Modelo de datos y diseño de componentes 9
      Diagramas UML 10
4. TECNOLOGÍAS EMPLEADAS 11
   4.1 Hardware necesario 11
   4.2 Software necesario 11
5. DESARROLLO DEL TRABAJO 13
6. CONCLUSIONES 13
7. REFERENCIAS 13
   7.1 Bibliografía 14
   7.2 Webgrafía 14
   7.3 Índice de citas e índice de imágenes 14
8. ANEXOS 14

1. ABSTRACT

Este proyecto propone el diseño e implementación de una plataforma de datos segura, automatizada y accesible para una empresa ficticia del sector e-commerce. A partir de un conjunto de datos realista, se aborda el ciclo completo de ingeniería de datos: desde la recogida de requisitos funcionales hasta la visualización de indicadores clave mediante dashboards interactivos. La solución integra tecnologías de virtualización, bases de datos relacionales, orquestación de flujos ETL y herramientas de Business Intelligence, todo ello desplegado sobre una infraestructura híbrida simulada y reforzada con medidas de seguridad operativas. El enfoque combina competencias del ciclo ASIR con principios de ingeniería de datos moderna, orientando el desarrollo hacia la utilidad empresarial y la sostenibilidad técnica.

El trabajo se ha estructurado en módulos que reflejan las competencias del ciclo ASIR y los principios de la ingeniería de datos moderna, con especial atención a la documentación, la automatización y la accesibilidad para usuarios no técnicos. Como resultado, se entrega una solución end-to-end que no solo cumple con los requisitos técnicos, sino que también aporta valor estratégico al negocio, demostrando la capacidad para diseñar sistemas útiles, sostenibles y alineados con las necesidades reales de las pymes.

2. INTRODUCCIÓN

2.1 Justificación y contexto

En un entorno empresarial cada vez más orientado al dato, las pymes se enfrentan al reto de aprovechar su información sin disponer de grandes recursos técnicos ni humanos.

La falta de automatización, seguridad operativa y accesibilidad para perfiles no técnicos limita su capacidad de análisis y toma de decisiones.

Este proyecto se enmarca en ese contexto, proponiendo una solución que simula un entorno realista de un e-commerce y que permite integrar infraestructura, seguridad y análisis de datos en una plataforma funcional y documentada.

2.2 Planteamiento del problema

Las pymes suelen operar con sistemas fragmentados, procesos manuales y escasa visibilidad sobre sus datos. Esto genera ineficiencias, dependencia de terceros y decisiones poco informadas.

El problema que se aborda es cómo diseñar una solución de datos que sea segura, automatizada y accesible, permitiendo a perfiles no técnicos consultar información útil sin comprometer la integridad ni la operatividad del sistema. Se parte de un dataset realista y se simula el desarrollo completo de una plataforma de datos, integrando tecnologías clave y buenas prácticas de documentación.

2.3 Objetivos del trabajo

El objetivo principal es construir una plataforma de datos que responda a las necesidades reales de una pyme del sector e-commerce, integrando competencias de ASIR con fundamentos de ingeniería de datos.

Objetivos específicos:

- Recoger y documentar requisitos funcionales del negocio.
- Diseñar una infraestructura híbrida segura y escalable.
- Automatizar la ingesta, transformación y carga de datos.
- Implementar una base de datos relacional optimizada para análisis.
- Configurar dashboards accesibles para usuarios no técnicos.
- Establecer mecanismos de monitorización y recuperación.
- Documentar cada fase con enfoque profesional y justificativo.

3. MOTIVACION

La elección de este proyecto responde al interés por construir soluciones tecnológicas que aporten valor real en entornos empresariales modestos.

En lugar de abordar un desarrollo aislado o centrado en una única herramienta, se ha optado por una solución completa que simula las necesidades de una pyme y permite aplicar conocimientos de infraestructura, seguridad, automatización y análisis de negocio.

Esta aproximación no solo permite demostrar habilidades técnicas, sino también capacidad de planificación, documentación y comunicación, elementos que considero clave para mi desarrollo profesional en el ámbito IT.

3.1 Marco teórico del trabajo

El presente proyecto se fundamenta en conceptos clave de la administración de sistemas y la ingeniería de datos, que permiten construir una plataforma segura, automatizada y accesible para una pyme del sector e-commerce.

En primer lugar, la virtualización constituye la base de la infraestructura. Mediante herramientas como VirtualBox y Vagrant se simulan máquinas físicas en un entorno controlado, lo que facilita la reproducibilidad y la portabilidad del sistema. Sobre estas máquinas virtuales se despliega Ubuntu Server, un sistema operativo estable y ampliamente utilizado en entornos corporativos.

La contenedorización con Docker aporta modularidad y aislamiento. Cada servicio (base de datos, ETL, visualización, monitorización) se ejecuta en su propio contenedor, garantizando independencia y escalabilidad.

El núcleo de la plataforma lo constituye PostgreSQL, un sistema gestor de bases de datos relacional robusto y de código abierto. Para reforzar la trazabilidad y el cumplimiento normativo se integra la extensión pgAudit, que permite registrar operaciones críticas y auditar el comportamiento de los usuarios.

El proceso de ETL (Extract, Transform, Load) se implementa mediante Python, utilizando librerías como Pandas para la manipulación de datos. Este proceso automatiza la ingesta desde fuentes externas, la transformación de los registros y su carga en la base de datos. Alternativamente, se contempla el uso de Pentaho PDI, que ofrece un enfoque visual y accesible para diseñar flujos de datos.

La visualización de la información se realiza con Metabase, herramienta de Business Intelligence que permite crear dashboards interactivos y accesibles para perfiles no técnicos. Complementariamente, Grafana se utiliza para monitorizar métricas técnicas y operativas, apoyándose en rsyslog para la recolección de logs.

En el ámbito de la seguridad operativa, se integran tecnologías como WireGuard (VPN ligera y segura), Suricata (sistema de detección de intrusiones), UFW (firewall de fácil configuración) y Fail2Ban (protección contra intentos de acceso no autorizados). Estas herramientas garantizan la resiliencia del sistema frente a amenazas externas.

3.2 Otros desarrollos anteriores

Se irán incluyendo a medida que se vayan haciendo dado que esto lo estoy escribiendo a 16 de noviembre de 2025, es decir, en la mayoría de las asignaturas aún estamos en el segundo tema.

3.3 Análisis funcional del proyecto

Este análisis funcional define con precisión qué hace la plataforma, para quién, cómo se estructura y qué requisitos debe cumplir. Incluye actores, funcionalidades, requisitos detallados, interfaces, modelo de datos y diagramas UML descritos y listos para incorporar.

Contexto, alcance y actores

Descripción de la pyme simulada

- **Sector:** E-commerce minorista.
- **Volumen de datos:** Ficheros diarios con registros de ventas y campañas.
- **Necesidad principal:** Unificar datos, automatizar su transformación y ofrecer dashboards seguros y accesibles.

Alcance del sistema

- **Incluye:** Ingesta vía API simulada, ETL (Python o Pentaho), base de datos PostgreSQL con pgAudit, BI con Metabase y monitorización con Grafana/rsyslog, seguridad operativa (UFW, Fail2Ban, WireGuard, Suricata).

- **No incluye:** Integración real con ERP/CRM externos, despliegue en cloud productivo, ML avanzado.

Actores y roles

- **Analista de negocio:** Consulta dashboards, descarga informes, solicita nuevas métricas.
- **Administrador de sistemas:** Despliega, configura seguridad, monitoriza y mantiene servicios.
- **Desarrollador de datos (ETL):** Diseña y ejecuta pipelines, valida calidad y esquema.
- **Proveedor externo de datos (API):** Expone endpoint JSON con ventas/campañas.
- **Base de datos (PostgreSQL + pgAudit):** Servicio que persiste y audita operaciones.
- **Herramientas de BI/Monitorización:** Metabase y Grafana como consumidores de datos.

Requisitos funcionales y no funcionales

Requisitos funcionales por actor

- **Analista de negocio:**
    - **Acceso a dashboards:** Visualizar ventas por periodo, producto y campaña.
    - **Filtros y segmentación:** Por fechas, categoría, canal de adquisición.
    - **Exportación:** Descarga de informes en CSV/PNG desde Metabase.
    - **Solicitudes de nuevas métricas:** Registrar requerimientos para el ETL.
- **Desarrollador de datos (ETL):**
    - **Ingesta de datos:** Consumir JSON desde API Flask y/o ficheros CSV.
    - **Transformaciones:** Limpieza, tipado, cálculo de métricas derivadas (total, margen).
    - **Validación:** Reglas de calidad (no nulos críticos, rangos, unicidad).
    - **Carga:** Insertar en tablas destino con idempotencia básica (evitar duplicados).
    - **Orquestación:** Lanzar tareas manualmente y programadas (cron).

  - **Trazabilidad:** Registrar logs de ejecución y resultados (volcados en rsyslog).

- **Administrador de sistemas:**
  - **Despliegue:** Provisionar VM/containers y la red interna segura.
  - **Seguridad perimetral:** Configurar UFW, Fail2Ban, WireGuard y Suricata (IDS).
  - **Disponibilidad:** Comprobaciones de salud y reinicio automático (restart policies).
  - **Backup/restore:** Copias de BD y configuración, pruebas de recuperación.
  - **Monitorización:** Dashboards técnicos, alertas por umbrales (CPU/RAM/espacio).
- **Base de datos (PostgreSQL + pgAudit):**
  - **Esquema relacional:** Tablas normalizadas para ventas, productos, campañas, clientes.
  - **Auditoría:** Registro de operaciones DML/DDL por usuario y timestamp.
  - **Roles y permisos:** Separación de lectura/escritura; acceso mínimo necesario.
- **Herramientas de BI/Monitorización:**
  - **Metabase:** Conexión a BD, creación de colecciones y permisos por rol.
  - **Grafana/rsyslog:** Ingesta de logs de contenedores/servicios; paneles de estado.

Requisitos no funcionales

- **Seguridad:** Principio de mínimo privilegio, cifrado en tránsito (VPN), firewall restrictivo.
- **Disponibilidad:** Reinicio automático de servicios, recuperación básica ante fallos.
- **Escalabilidad:** Diseño modular con contenedores; posibilidad de migración a cloud.
- **Mantenibilidad:** Configuración declarativa (docker-compose), documentación y logs.
- **Trazabilidad:** Auditoría completa de operaciones en BD; logs centralizados.

- **Rendimiento:** ETL diario < 5 minutos para dataset simulado (escenario pyme).
- **Usabilidad:** Dashboards claros, filtros intuitivos, exportación sencilla.

Funcionalidades detalladas y criterios de aceptación

- **Ingesta de ventas desde API:**
  - **Criterio:** El ETL realiza GET a /api/ventas, recibe JSON válido (HTTP 200), persiste al menos el 95% de registros tras validación.
- **Transformación de datos:**
  - **Criterio:** Campos tipados correctamente; se calcula total = cantidad × precio_unitario; fechas en ISO; categorías normalizadas.
- **Carga en PostgreSQL:**
  - **Criterio:** Inserciones con control de duplicados (unique key por fecha+producto+id_venta cuando aplique); transacción atómica por lote.
- **Auditoría con pgAudit:**
  - **Criterio:** Toda operación INSERT/UPDATE/DELETE queda registrada con usuario, tabla, timestamp; consulta de auditoría accesible para revisión.
- **Dashboards en Metabase:**
  - **Criterio:** Al menos 3 dashboards operativos (ventas por periodo, top productos, campañas); filtros funcionales; exportación habilitada.
- **Monitorización técnica:**
  - **Criterio:** Panel con estado de contenedores, CPU/RAM, espacio; alerta cuando uso de disco > 80% o servicio no responde.
- **Seguridad operativa:**
  - **Criterio:** UFW activo con puertos mínimos; Fail2Ban bloquea intentos de fuerza bruta; acceso externo solo vía WireGuard; Suricata genera alertas en pruebas.

Interfaces del sistema y contrato de datos

Interfaces externas

- **API Flask proveedor:**
  - **Endpoint:** GET /api/ventas

  - **Autenticación:** Simulada (clave API simple o sin auth en entorno académico).

- **Metabase (usuario analista):**
  - **Acceso: HTTP/HTTPS a interfaz web.**
  - **Operaciones: Visualizar dashboards, filtrar, exportar.**

Interfaces internas

- **ETL → PostgreSQL:**
  - **Operación:** INSERT/UPSERT a tablas destino; transacciones por lote.
- **pgAudit → Logs:**
  - **Salida:** Registros en vistas o logs del servidor PostgreSQL.
- **Servicios → rsyslog:**
  - **Canal:** Logs de contenedores y servicios centralizados para Grafana.

Modelo de datos y diseño de componentes

Esquema de base de datos (visión lógica)

- **Dimensiones:**
  - **dim_producto:** id_producto, nombre, categoría.
  - **dim_campania:** id_campania, nombre, canal, fecha_inicio, fecha_fin.
  - **dim_fecha:** fecha, año, mes, día, semana.
- **Hechos:**
  - **fact_ventas:** id_venta, fecha, id_producto, id_campania, cantidad, precio_unitario, total.
- **Claves y restricciones:**
  - **PKs:** id_venta en fact_ventas; ids en dimensiones.
  - **FKs:** fact_ventas.id_producto → dim_producto.id_producto; fact_ventas.id_campania → dim_campania.id_campania.
  - **Índices:** por fecha, producto y campaña; unique por combinación clave de negocio si aplica.

Componentes del sistema

- **Proveedor de datos (API Flask):** Emula fuente externa REST.
- **ETL (Python o Pentaho):** Extrae, transforma y carga; registra logs.
- **PostgreSQL + pgAudit:** Persistencia y auditoría de operaciones.
- **Metabase:** BI para analistas con permisos por rol.
- **Grafana + rsyslog:** Monitorización, alertas y evidencias operativas.
- **Seguridad (UFW, Fail2Ban, WireGuard, Suricata):** Protección perimetral, VPN y detección.

Diagramas UML

Diagrama de casos de uso

- **Actores:** Analista, Desarrollador ETL, Administrador Sistemas.
- **Casos:**
    - **Analista:** Consultar dashboards, filtrar, exportar.
    - **Desarrollador ETL:** Ejecutar ETL, validar calidad, cargar datos.
    - **Administrador:** Desplegar servicios, configurar seguridad, monitorizar, backup/restore.

Diagrama de componentes

- **Componentes:** API Flask, ETL, PostgreSQL+pgAudit, Metabase, Grafana/rsyslog, Seguridad (UFW/Fail2Ban/WireGuard/Suricata).

Diagrama de secuencia (flujo ETL diario)

- **Escenario:** Ejecución programada, ingesta desde API, transformación y carga, auditoría y notificación.

Diagrama de despliegue (alto nivel)

- **Nodos:** VM Ubuntu Server, contenedores Docker, red interna, acceso VPN.

Trazabilidad entre requisitos y módulos

- **Ingesta y transformación (Módulo ETL):** Cumple RF de

extracción/transformación/carga y RNF de rendimiento y trazabilidad.

- **Persistencia y auditoría (BD + pgAudit):** Responde a RF de almacenamiento y RNF de seguridad/tra zabilidad.
- **Visualización (Metabase):** Direcciona RF de acceso para analistas y RNF de usabilidad.
- **Seguridad y monitorización:** Asegura RNF de seguridad, disponibilidad y mantenibilidad.
- **Automatización y mantenimiento:** Garantiza RNF de continuidad operativa y documentación.

4. TECNOLOGÍAS EMPLEADAS

4.1 Hardware necesario

El proyecto se despliega sobre una infraestructura simulada que reproduce las necesidades de una pyme real, pero adaptada a un entorno académico. Se utiliza una máquina virtual con recursos modestos (2–4 GB de memoria RAM, 2 núcleos de CPU y 20 GB de almacenamiento), suficientes para ejecutar los servicios de forma eficiente.

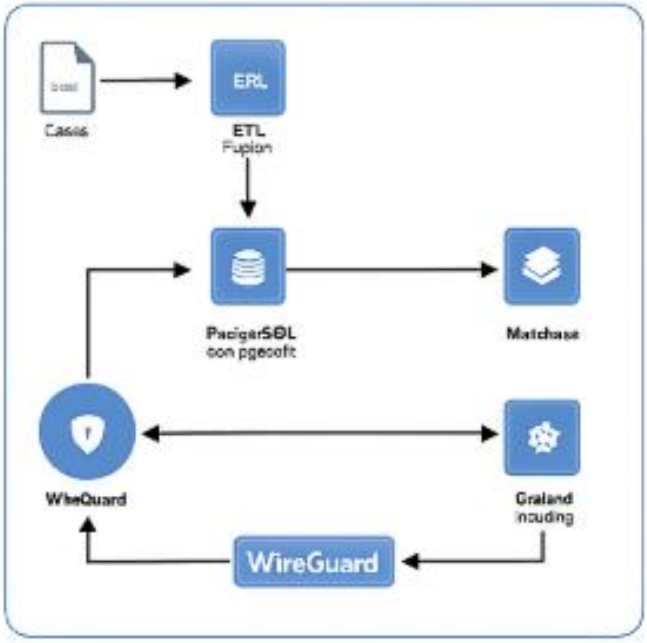
La elección de este hardware responde a criterios de sostenibilidad y realismo: una pyme con recursos limitados puede operar con servidores de gama media, y la modularidad del sistema permite escalar hacia soluciones cloud si se requiere mayor capacidad. Además, la separación lógica de servicios en contenedores simula un entorno distribuido, aunque físicamente se ejecute en una única máquina.

4.2 Software necesario

La selección de software se ha realizado siguiendo criterios de aplicabilidad profesional, sostenibilidad técnica y disponibilidad de soporte comunitario.

- **PostgreSQL** se justifica por su robustez, su carácter open source y su amplia adopción en entornos empresariales.
- **pgAudit** añade trazabilidad y auditoría, aspectos críticos para cumplir con normativas de seguridad y protección de datos.
- **Python** se emplea para el proceso ETL por su flexibilidad, potencia en el manejo de datos y facilidad de automatización. Se contempla **Pentaho PDI** como alternativa visual, especialmente útil para usuarios no técnicos.
- **Metabase** se selecciona como herramienta de BI por su sencillez, accesibilidad y capacidad de generar dashboards interactivos sin necesidad de conocimientos avanzados.
- **Grafana** y **rsyslog** permiten monitorizar el sistema y visualizar métricas técnicas, reforzando la resiliencia operativa.

- En el ámbito de la seguridad, **WireGuard**, **Suricata**, **UFW** y **Fail2Ban** ofrecen un conjunto completo de mecanismos de protección, desde el acceso remoto seguro hasta la detección de intrusiones y la mitigación de ataques.
- Finalmente, **VirtualBox y Vagrant** proporcionan un entorno reproducible y portable, ideal para el despliegue académico y la simulación de escenarios reales.



5. DESARROLLO DEL TRABAJO

En este capítulo se describen los trabajos desarrollados por el propio alumno a partir de los puntos descritos previamente. La estructura de este capítulo depende mucho del tipo de trabajo realizado, debiendo adaptarse a éste.

Es la parte más importante del PFC, y debería ser, por tanto, la desarrollada con mayor amplitud y detalle.

6. CONCLUSIONES

Primará la claridad de exposición, el rigor en el desarrollo de las ideas y la concreción de la escritura. Junto con el capítulo de Introducción, al que debe dar respuesta, este capítulo debería poder leerse con autonomía del resto del documento. Debe incluir sugerencias sobre trabajos futuros que pudiesen continuar con el trabajo iniciado en el PFC.

7. REFERENCIAS

Todas las imágenes, citas al pie y referencias del PFC han de quedar registradas en este apartado siguiendo la normativa APA.

7.1 Bibliografía

Texto correspondiente

7.2 Webgrafía

Texto correspondiente

7.3 Índice de citas e índice de imágenes

En caso de incluir citas, imágenes o ilustraciones, el PFC deberá contar con los respectivos índices de citas, imágenes o ilustraciones referidos en el sistema de citas.

Las imágenes e ilustraciones deben estar acompañadas de un pie de foto o subtítulo que especifique su empleo en el texto. Imágenes y citas deben ser numeradas por separado según un registro que debe quedar resumido en este apartado.

En caso de imágenes no creadas por el autor del PFC bajo el pie de foto deben aparecer la página y el libro o url de donde se ha extraído la imagen.

8. ANEXOS

En los anexos se adjuntará todo el material citado que ocupe demasiado espacio para ser incluido en el cuerpo de texto o que pueda desviar la atención del lector.

Cualquier gráfico, estadística, línea de código, texto de referencia, entrevista o escolio técnico que no esté directamente vinculado con el desarrollo y las conclusiones del PFC pero que se quiera añadir como material de consulta es susceptible de incluirse como anexo.